Performance Comparison of 2D Convolution Using SIMD

ECE 662 – Laboratory Report

Abishek Thirunavukkarasu

November 2026

Implementation: Intel AVX2 SIMD

Abstract

This laboratory exercise investigates the performance improvement obtained by implementing 2D convolution using Intel AVX2 SIMD instructions.

A conventional nested-loop implementation is compared against an AVX2-optimized implementation written in C. The experiment uses a 2048×2048 input matrix and a 4×4 convolution kernel.

The AVX2 implementation successfully passes correctness verification across five experimental runs. The measured average execution time improves from 0.0704 seconds to 0.0536 seconds, achieving approximately $1.31\times$ speedup.

1 Objective

The objective of this laboratory exercise is to:

- Understand and implement 2D convolution in C.
- Compare standard and AVX-optimized 2D convolution implementations.
- Learn to use Intel SIMD instructions for integer operations.
- Evaluate the performance improvement obtained through SIMD optimization.

2 Experimental Setup

The experiment was implemented in C using GCC on a Windows environment.

The CPU was checked for SIMD instruction-set support. The compiler detected AVX2 support.

Parameter	**Configuration**
Programming Language	C
Compiler	GCC
Operating Environment	Windows / PowerShell
SIMD Instruction Set	AVX2
Optimization Level	-O3
SIMD Compiler Flag	-mavx2

Table 1: Experimental software configuration

The program was compiled using:

```
gcc -std=c11 -O3 -mavx2 -o conv2d conv2d.c
```

3 Input Configuration

The following dimensions were used in the experiment:

Parameter	**Value**
Input Height	2048
Input Width	2048
Kernel Height	4
Kernel Width	4
Input Data Type	int8
Output Data Type	int32
Output Height	2045
Output Width	2045

Table 2: Input and kernel configuration

The input matrix and kernel were initialized with random integer values.
The output dimensions are calculated as:

$$O_H = I_H - K_H + 1$$

$$O_W = I_W - K_W + 1$$

Therefore:

$$O_H = 2048 - 4 + 1 = 2045$$

$$O_W = 2048 - 4 + 1 = 2045$$

4 Implementation

4.1 Naive 2D Convolution

The naive implementation uses nested loops to traverse the input matrix and apply the 4×4 kernel.

For every output pixel, corresponding input and kernel elements are multiplied and accumulated.

Each output pixel requires:

$$4 \times 4 = 16$$

multiplications.
The total number of output pixels is:

$$2045 \times 2045 = 4,182,025$$

Therefore, the total number of scalar multiplications is:

$$4,182,025 \times 16 = 66,912,400$$

Approximately 66.9 million scalar multiplications are required.

4.2 AVX2 SIMD Optimization

The AVX2 implementation uses Intel SIMD intrinsics to perform multiple integer operations in parallel.

The implementation follows these steps:

1. Load exactly four input bytes and four kernel bytes into 128-bit registers.
2. Convert signed 8-bit integers into signed 16-bit integers using `_mm256_cvtepi8_epi16`.
3. Perform element-wise multiplication using `_mm256_mullo_epi16`.
4. Store the multiplication results using `_mm256_storeu_si256`.
5. Accumulate the first four valid multiplication results to calculate the output pixel.

The implementation uses four meaningful SIMD lanes for each kernel row. The remaining lanes are zero.

Thus, four meaningful multiplication operations are performed using one vector multiplication instruction.

5 Correctness Verification

The output of the AVX2 implementation was compared against the output of the naive implementation.

The comparison checks every output element for equality.

The program returned:

```
Correct: 1
```

The correctness result was consistently 1 across all five experimental runs.

This confirms that the AVX2 implementation produces the same output as the naive implementation for the tested inputs.

6 Performance Evaluation

The execution time of both implementations was measured over five independent program executions.

Run	Naive (s)	AVX2 (s)	Correct
1	0.067	0.053	1
2	0.067	0.052	1
3	0.070	0.053	1
4	0.079	0.056	1
5	0.069	0.054	1
Average	**0.0704**	**0.0536**	**1**

Table 3: Runtime comparison between naive and AVX2 convolution

6.1 Average Execution Time

The average execution time is calculated as:

$$T_{\text{avg}} = \frac{\sum_{i=1}^{5} T_i}{5}$$

For the naive implementation:

$$T_{\text{naive}} = \frac{0.067 + 0.067 + 0.070 + 0.079 + 0.069}{5}$$

$$\boxed{T_{\text{naive}} = 0.0704 \text{ seconds}}$$

For the AVX2 implementation:

$$T_{\text{AVX2}} = \frac{0.053 + 0.052 + 0.053 + 0.056 + 0.054}{5}$$

$$\boxed{T_{\text{AVX2}} = 0.0536 \text{ seconds}}$$

6.2 Speedup Calculation

The speedup is calculated using:

$$\text{Speedup} = \frac{T_{\text{naive}}}{T_{\text{AVX2}}}$$

Substituting the measured average execution times:

$$\text{Speedup} = \frac{0.0704}{0.0536}$$

$$\boxed{\text{Speedup} \approx 1.31\times}$$

6.3 Runtime Reduction

The percentage reduction in execution time is:

$$\text{Reduction} = \frac{T_{\text{naive}} - T_{\text{AVX2}}}{T_{\text{naive}}} \times 100$$

$$= \frac{0.0704 - 0.0536}{0.0704} \times 100$$

$$\boxed{\text{Runtime Reduction} \approx 23.86\%}$$

7 Experimental Execution Output

The following screenshot shows the actual execution results of the AVX2 implementation across five runs.

All five runs successfully passed correctness verification, with the measured execution times shown in the terminal output.

```
PS C:\Users\abish> gcc -std=c11 -O3 -mavx2 -o conv2d conv2d.c
PS C:\Users\abish> .\conv2d.exe
Correct: 1
Naive: 0.067 sec
AVX:   0.054 sec
PS C:\Users\abish> 1..5 | ForEach-Object {
>>     Write-Host "Run $_"
>>     .\conv2d.exe
>> }
Run 1
Correct: 1
Naive: 0.067 sec
AVX:   0.053 sec
Run 2
Correct: 1
Naive: 0.067 sec
AVX:   0.052 sec
Run 3
Correct: 1
Naive: 0.070 sec
AVX:   0.053 sec
Run 4
Correct: 1
Naive: 0.079 sec
AVX:   0.056 sec
Run 5
Correct: 1
Naive: 0.069 sec
AVX:   0.054 sec
PS C:\Users\abish>
```

Figure 1: PowerShell execution output showing correctness verification and runtime measurements across five experimental runs.

8 Discussion

The AVX2 implementation achieved a measurable performance improvement compared with the naive implementation.

The average execution time decreased from 0.0704 seconds to 0.0536 seconds, resulting in approximately 1.31 times speedup.

Although AVX2 allows multiple integer operations to be performed using a single vector instruction, the overall execution time also depends on memory access, data conversion, storing intermediate results, accumulation, and loop overhead.

Therefore, the reduction in execution time is not necessarily proportional to the number of SIMD lanes.

All five experimental runs returned a correctness result of 1, demonstrating consistent functional equivalence between the two implementations.

9 Conclusion

This laboratory exercise demonstrated the use of Intel AVX2 SIMD intrinsics to optimize 2D convolution in C.

The naive implementation was compared against an AVX2-optimized implementation using a 2048×2048 input matrix and a 4×4 kernel.

The AVX2 implementation successfully passed all five correctness checks.

The average execution time improved from 0.0704 seconds to 0.0536 seconds, achieving approximately 1.31 times speedup and a 23.86% reduction in execution time.

The experiment illustrates how SIMD instructions can improve computational performance by processing multiple data elements simultaneously.